

M17 - Architecture + Promotion Foundations

Zero-to-Job-Ready Data Engineer 2026 | RC2 | 10 architecture lessons

ARCHITECTURE RULE - Requirements first, tools second. Every design must explain trade-offs, failures, security, cost and operability.

M17-L01 - From Requirements to Architecture

Why this matters

Architecture starts with requirements, not favorite tools. A junior engineer should learn to extract the facts that materially change the design.

Start with the simple idea

Translate a vague request into constraints: data sources, volume, velocity, latency, correctness, retention, consumers, security, availability, budget and team capability.

Technical model

- Functional requirements describe what the platform must do.
- Non-functional requirements describe scale, latency, availability, security, cost and operability.
- Separate hard constraints from preferences.
- Record assumptions explicitly; unknowns are not facts.
- Every architecture choice should trace back to at least one requirement.

Worked reasoning example

A product team says “real-time dashboard.” When questioned, they only need hourly updates. That changes the design from always-on streaming to a simpler scheduled batch/micro-batch system with lower operational cost.

Evidence to produce

- Requirements table with source/volume/latency/consumer/security/cost.
- Assumption list.
- Decision trace from requirement to component.

Decision framework

- Write a one-line outcome: what decision/action will the data enable?
- Quantify latency instead of words such as fast or real-time.
- Separate steady-state volume from bursts and growth.
- Identify data classification/retention constraints before storage choice.
- Record team/on-call ownership because operability is a requirement too.

Anti-patterns to avoid

- Tool-first diagram with no measurable SLA.
- Assumption presented as fact.
- One “platform” requirement that mixes BI, alerting and data science into the same latency path.

Mental model

SOURCE requirements -> constraints/assumptions -> candidate patterns -> trade-off review -> architecture decision

Review question

What requirement justifies this choice, what is the strongest alternative, and what evidence would make you change your mind?

M17-L02 - Batch vs Streaming Decisions

Why this matters

Streaming adds continuous operational complexity. It should be chosen for business latency, not because it sounds more advanced.

Start with the simple idea

Batch processes bounded datasets at intervals. Streaming continuously processes unbounded events. Micro-batch sits between those extremes.

Technical model

- Choose batch when minutes/hours latency is acceptable and sources are naturally periodic.
- Choose streaming when the business outcome loses value if delayed beyond seconds/minutes.
- Consider event ordering, replay, duplicates, late events and 24x7 operation.
- Streaming may still feed batch analytical models downstream.
- Do not confuse ingestion frequency with dashboard refresh requirement.

Worked reasoning example

A fraud-alert workflow needs action within 10 seconds, so streaming is justified. A daily finance close can remain batch even if the source system emits events continuously.

Evidence to produce

- Latency requirement stated numerically.
- Business consequence of delay.
- Operational costs/risks of streaming acknowledged.

Decision framework

- Start from maximum acceptable delay.
- Ask whether the source naturally arrives continuously or periodically.
- List streaming-specific responsibilities: ordering, replay, duplicates, lag and always-on monitoring.
- Compare business value of lower latency with added operational burden.
- Use separate paths when one domain has both second-level alerts and daily reporting.

Anti-patterns to avoid

- “Use Kafka because volume is large.” Volume alone does not require streaming.
- Streaming every downstream transformation when only ingestion needs low latency.
- Ignoring late/out-of-order events in the design.

Mental model

Events -> streaming alert path (<30s)

Events/files -> batch/micro-batch analytical path (hourly/daily)

Review question

What requirement justifies this choice, what is the strongest alternative, and what evidence would make you change your mind?

M17-L03 - Warehouse vs Lakehouse

Why this matters

Warehouse and lakehouse are architectural patterns optimized for overlapping but different workloads.

Start with the simple idea

Choose the simplest platform that satisfies consumer, data-type, governance and compute needs. Hybrid patterns are normal.

Technical model

- Warehouse strengths: SQL-first structured analytics, BI concurrency, mature relational controls.
- Lakehouse strengths: files + tables, Spark/distributed processing, mixed structured/semi-structured data, open table formats.
- A lakehouse is not a dumping ground; it still needs contracts, layers, quality and ownership.
- A warehouse can coexist with a lakehouse when consumers have different needs.

Worked reasoning example

A team stores clickstream JSON, large Parquet history and ML features while also publishing curated BI marts. A lakehouse for scalable processing plus SQL-facing curated models may fit better than forcing every raw file directly into a warehouse.

Evidence to produce

- Workload table.
- Chosen primary pattern + trade-offs.
- Consumer access path.

Decision framework

- List data types and dominant query/compute engines.
- List consumers: BI, ad-hoc SQL, Spark/data science, external sharing.
- Choose where authoritative curated models live.
- Define raw/landing retention and governance.
- Consider hybrid only when the two patterns serve distinct requirements.

Anti-patterns to avoid

- Calling a folder of unmanaged files a lakehouse.
- Moving all raw clickstream into an operational database.
- Adding both warehouse and lakehouse without defining system-of-record boundaries.

Mental model

RAW files/events -> Lakehouse/Spark processing -> curated tables -> SQL/BI
(optional dedicated warehouse marts when concurrency/governance needs justify it)

Review question

What requirement justifies this choice, what is the strongest alternative, and what evidence would make you change your mind?

M17-L04 - Managed Service vs Build Yourself

Why this matters

Managed services trade some control for less operational work. Self-managed systems trade more responsibility for flexibility.

Start with the simple idea

Architecture includes the team that will operate it. A technically possible design can still be a bad organizational fit.

Technical model

- Managed services reduce patching/control-plane burden but may increase platform dependence or unit cost.
- Self-managed systems require upgrades, backups, scaling, monitoring and incident expertise.
- Choose based on required control, skills, compliance and total operating cost.
- Prefer boring/reliable operations for small teams unless a clear requirement demands custom control.

Worked reasoning example

A three-person data team can run a managed warehouse and orchestration service reliably; self-hosting a complex cluster may consume more engineering time than it saves in infrastructure cost.

Evidence to produce

- Responsibility split.
- Team-skill fit.
- Exit/vendor-dependence risk.

Decision framework

- Create a responsibility table for upgrades, patching, scaling, backups/control-plane and incident response.
- Estimate how much specialist time the team can sustainably own.
- Identify requirements that truly need low-level control.
- Include migration/lock-in risk in the decision rather than pretending it does not exist.

Anti-patterns to avoid

- Comparing only sticker price and ignoring engineer/on-call time.
- Self-hosting because it feels more senior.
- Managed service chosen without understanding service limits/quotas.

Mental model

Managed: provider owns more platform operations

Self-managed: team owns more platform operations + flexibility

Review question

What requirement justifies this choice, what is the strongest alternative, and what evidence would make you change your mind?

M17-L05 - Scaling and Bottleneck Reasoning

Why this matters

Scale problems are rarely solved by “make everything bigger.” First identify what resource or stage is limiting throughput.

Start with the simple idea

Measure before optimizing. Separate source, network, compute, shuffle, storage, metadata/small-file and downstream bottlenecks.

Technical model

- Volume: total bytes/rows/events.
- Velocity: arrival rate and burstiness.
- Parallelism: how work is partitioned.
- Data movement: joins/shuffles/network egress.
- Small-file/metadata overhead can dominate even when total data is modest.
- Scaling one stage may move the bottleneck elsewhere.

Worked reasoning example

A Spark job slows after a new join. CPU is not saturated, but shuffle bytes explode and one key owns 40% of rows. The first fix is skew/join design, not simply doubling cluster size.

Evidence to produce

- Baseline metrics.
- Bottleneck hypothesis tied to evidence.
- One targeted change and expected effect.

Decision framework

- Capture baseline volume, duration, partition/file counts and stage timings.
- Find the first resource/stage that changed with the slowdown.
- Form one falsifiable hypothesis.
- Change one factor and compare evidence.
- Re-evaluate because fixing one bottleneck may reveal the next.

Anti-patterns to avoid

- Doubling every cluster setting.
- Optimizing code before proving where time is spent.
- Ignoring skew/small-file/network effects because CPU looks low.

Mental model

measure -> identify bottleneck -> hypothesis -> targeted change -> compare -> repeat

Review question

What requirement justifies this choice, what is the strongest alternative, and what evidence would make you change your mind?

M17-L06 - Reliability and Failure Domains

Why this matters

Reliable systems assume components and dependencies can fail. The design limits blast radius and makes recovery predictable.

Start with the simple idea

Ask “what if this fails?” for source, network, transform, storage, scheduler, credentials and downstream publication.

Technical model

- Idempotency makes retries safe.
- Checkpoint/watermark state must advance only after success.
- Separate raw retention from curated publication so reprocessing is possible.
- Design backfill/replay before the first incident.
- Failure domains and dependency isolation reduce blast radius.
- Define acceptable recovery time and data loss conceptually (RTO/RPO awareness).

Worked reasoning example

An API extraction succeeds but the target write fails. If the watermark already advanced, records are lost. Reliability design commits the watermark only after durable successful load and preserves raw input for replay.

Evidence to produce

- Failure-mode table.
- Recovery path.
- Validation after recovery.

Decision framework

- List each external dependency and state what “unavailable” looks like.
- Define retryable vs non-retryable failures.
- State where checkpoints/watermarks commit.
- State what raw/history evidence allows replay.
- Define validation after recovery so repaired data is proven correct.

Anti-patterns to avoid

- Retry without idempotency.
- Advancing state before target durability.
- Backfill process invented during the incident.

Mental model

detect -> contain -> retry/replay/backfill -> validate -> publish

Review question

What requirement justifies this choice, what is the strongest alternative, and what evidence would make you change your mind?

M17-L07 - Cost-Aware Architecture

Why this matters

Cloud cost is an engineering signal, not only a finance problem. Architecture determines how often and how much compute, storage and data movement you pay for.

Start with the simple idea

Estimate the major cost drivers before optimizing pennies: always-on compute, scanned data, shuffle, storage copies, egress and over-frequent runs.

Technical model

- Storage cost is usually lower than compute but unnecessary copies accumulate.
- Serverless/on-demand can fit bursty workloads; always-on capacity can fit steady utilization.
- Partition pruning and incremental processing reduce scanned work.
- Streaming has an always-on operational cost even at low traffic.
- Measure cost per useful workload/output, not only monthly total.

Worked reasoning example

A job reads 10 TB daily to process 20 GB of changes. Incremental design can reduce compute and scan cost far more than micro-optimizing Python syntax.

Evidence to produce

- Cost-driver list.
- One high-impact optimization.
- Trade-off: cost vs latency/reliability/simplicity.

Decision framework

- Rank compute, scanned data, storage copies, egress and always-on services by expected impact.
- Normalize by workload: cost per daily refresh, TB processed or data product.
- Find waste before reducing resilience.
- Use incremental processing/partition pruning when the change fraction is small.
- Treat latency as a cost variable: faster usually requires more continuously available resources.

Anti-patterns to avoid

- Cutting monitoring/backup to save small amounts.
- Keeping an always-on streaming stack for daily-only output.
- Optimizing storage pennies while scanning terabytes unnecessarily.

Mental model

business SLA -> workload pattern -> cost drivers -> optimization -> recheck SLA/reliability

Review question

What requirement justifies this choice, what is the strongest alternative, and what evidence would make you change your mind?

M17-L08 - Security and Governance by Design

Why this matters

Security and governance are architecture constraints, not stickers added after the pipeline is built.

Start with the simple idea

Design identity, access boundaries, sensitive-data flow, encryption, ownership, lineage and audit evidence alongside the data path.

Technical model

- Use workload identities and least privilege.
- Classify sensitive fields and minimize copies.
- Separate environments and production access.
- Define ownership and lineage for critical products.
- Keep secrets out of code and logs.
- Design audit/monitoring evidence for privileged changes.

Worked reasoning example

A raw customer zone contains PII. Curated consumer marts should not inherit raw access automatically. The architecture separates restricted raw storage from curated data and gives each identity only the required scope.

Evidence to produce

- Identity/action/scope matrix.
- Sensitive-data flow.
- Owner/lineage/audit controls.

Decision framework

- Draw trust boundaries and classify data at entry.
- List identities separately: human, pipeline/workload, platform admin.
- Give each identity required actions at minimum scope.
- Define which sensitive fields cross into curated zones.
- Attach owner/lineage/audit requirements to critical outputs.

Anti-patterns to avoid

- One shared admin identity for everything.
- Raw-data permissions automatically inherited by curated consumers.
- Governance document that is not connected to access or quality controls.

Mental model

restricted RAW -> governed processing identity -> minimized CURATED -> consumer-specific read access

Review question

What requirement justifies this choice, what is the strongest alternative, and what evidence would make you change your mind?

M17-L09 - Promotion Foundations: CDC, Contracts, Vault, Mesh, Kubernetes

Why this matters

A strong junior should recognize adjacent patterns without pretending to have production mastery.

Start with the simple idea

Learn the problem each advanced concept solves, the trade-off it introduces and when deeper study becomes useful.

Technical model

- CDC captures source changes continuously or near-continuously; it introduces ordering, deletes and schema concerns.
- Data contracts formalize producer/consumer expectations such as schema, semantics, ownership and compatibility.
- Data Vault is a warehouse modeling approach emphasizing historization, source traceability and scalable integration.
- Data Mesh is an organizational/architecture approach emphasizing domain ownership, data products and federated governance.
- Kubernetes orchestrates containers at scale; it is not required simply because Docker exists.
- Deeper Terraform/platform engineering becomes valuable when infrastructure ownership grows.

Worked reasoning example

A small team with one warehouse does not need Data Mesh merely because it is modern terminology. A company with many autonomous domains and shared platform/governance challenges may benefit from its principles.

Evidence to produce

- Correct one-sentence problem statement for each concept.
- One “use later” and one “do not use yet” example.

Decision framework

- For each concept, name the problem first.
- State the organizational/operational cost it introduces.
- State the trigger that would justify deeper implementation study.
- Keep “awareness” separate from “production mastery.”

Anti-patterns to avoid

- Adding Data Mesh to a one-team data platform.
- Using Kubernetes because containers exist.
- Using CDC without a delete/update/order strategy.
- Using Data Vault without a real integration/historization need.

Mental model

Problem -> pattern candidate -> trade-offs -> proof of need -> deeper specialization

Review question

What requirement justifies this choice, what is the strongest alternative, and what evidence would make you change your mind?

M17-L10 - Architecture Review and Defense

Why this matters

Architecture quality improves when assumptions and trade-offs can survive review by another engineer.

Start with the simple idea

A review is not about drawing the prettiest diagram. It tests requirements coverage, failure behavior, security, cost, operability and alternatives.

Technical model

- State requirements and assumptions first.
- Show data flow and trust boundaries.
- Explain why each major component exists.
- Name top failure modes and recovery paths.
- Name top cost/scaling risks.
- Present at least one reasonable alternative and why it was not chosen.
- Record open questions rather than hiding uncertainty.

Worked reasoning example

Two designs both work technically. Design A has lower latency but doubles operational services; Design B meets the actual 30-minute SLA with batch and is simpler. A good review can prefer B and defend the choice.

Evidence to produce

- Architecture decision record.
- Risk register.
- Alternative analysis.
- Clear answers to “what changes at 10x?” and “what fails first?”.

Decision framework

- Present requirements and assumptions before the diagram.
- Walk source -> processing -> storage -> consumers and explain each boundary.
- Name top three failure modes and recovery paths.
- Name top two scaling/cost risks.
- Present the strongest alternative and why it lost.
- Answer what changes at 10x and what evidence would trigger redesign.

Anti-patterns to avoid

- Defending every original choice instead of responding to evidence.

- Architecture diagram with no owners/recovery/security.
- Claiming “scalable” without identifying a scaling dimension or limit.

Mental model

requirements -> design -> risks -> alternatives -> review questions -> recorded decision

Review question

What requirement justifies this choice, what is the strongest alternative, and what evidence would make you change your mind?